

Algorithmen und Datenstrukturen

A top-down view of a person sitting at a desk, working on a computer. The desk is illuminated by three large, angled monitors. The person is wearing a grey t-shirt and is using a keyboard and a mouse. A white mug is on the desk to the left of the keyboard. A skateboard is on the floor to the right of the desk. The background is dark.

Ziele der heutigen Lerneinheit

Sie verstehen...

- was ein Graph ist und wie gerichtete und ungerichtete Graphen sich unterscheiden
- wie BFS vorgeht und warum sie den kürzesten Pfad (in Kantenzahl) findet
- wie DFS vorgeht und was die Zeitmarken aussagen

Sie können...

- einen Graphen als Adjazenzliste oder Adjazenzmatrix darstellen
- BFS und DFS schrittweise auf einem gegebenen Graphen ausführen und die Knoten-/Kantenfärbung nachvollziehen
- eine topologische Sortierung mithilfe der DFS-Endzeiten bestimmen

Graphen

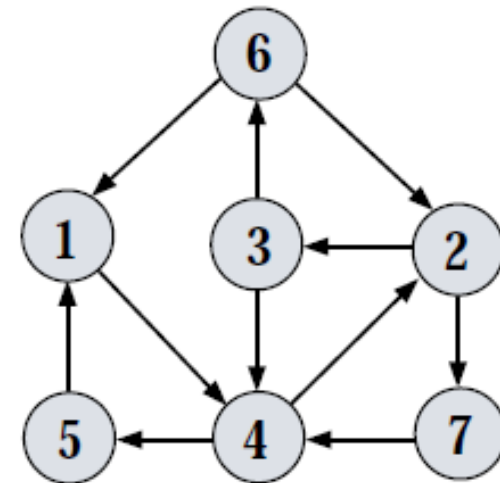
Graphen

Gerichtete Graphen

Ein gerichteter Graph $G=(V, E)$ besteht aus

- einer endlichen Menge von Knoten V (vertices)
- einer endlichen Menge E von geordneten Paaren aus V , den Kanten (edges)

Ist $(q, r) \in E$ eine Kante, so ist $q \in V$ der Anfangsknoten und $r \in V$ der Endknoten dieser Kante.



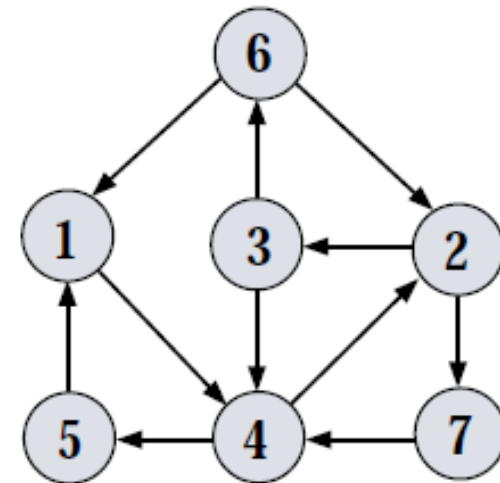
Graphen

Gerichtete Graphen

Im Beispiel:

$G=(V, E)$ mit

- $V=\{1, 2, 3, 4, 5, 6, 7\}$
- $E=\{(1,4), (2,3), (2,7), (3,4), (3,6), (4,2), (4,5), (5,1), (6,1), (6,2), (7,4)\}$

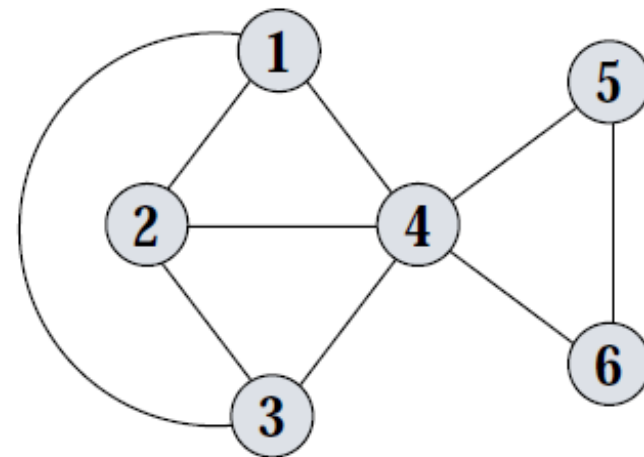


Graphen

Ungerichtete Graphen

Ein ungerichteter Graph $G=(V, E)$ besteht aus

- einer endlichen Menge von Knoten V (vertices)
- einer endlichen Menge E von ungeordneten Paaren aus V , den Kanten (edges)



Graphen

Wichtige Eigenschaften

- Ist $(q, r) \in E$ eine Kante eines Graphen G , so ist der Knoten r **adjazent** zum Knoten q . Ist G ungerichtet, dann ist auch q **adjazent** zum Knoten r
- Ein Graph G , in dem jeder Knoten mit jedem anderen Knoten adjazent ist, heißt **vollständiger Graph**.
- Ein Graph G heißt **dicht (dense)**, wenn er nahezu vollständig ist.
- Ein Graph G heißt **licht (sparse)**, wenn die Kantenanzahl viel kleiner als $|V|^2$ ist.

Graphen

Datenstrukturen

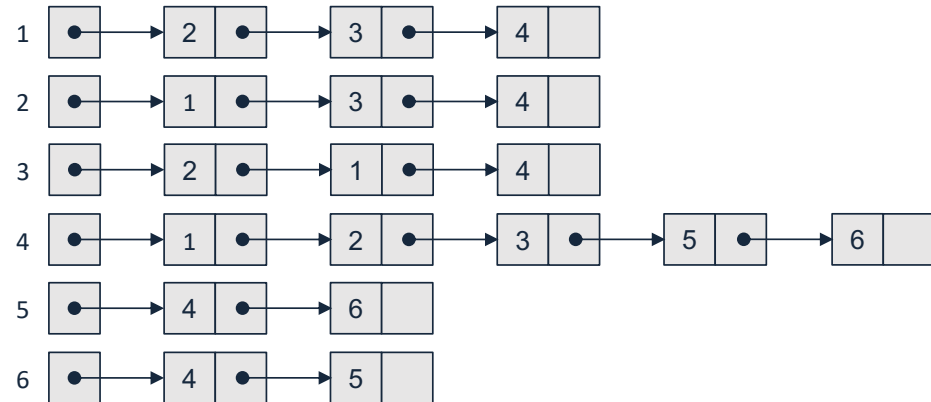
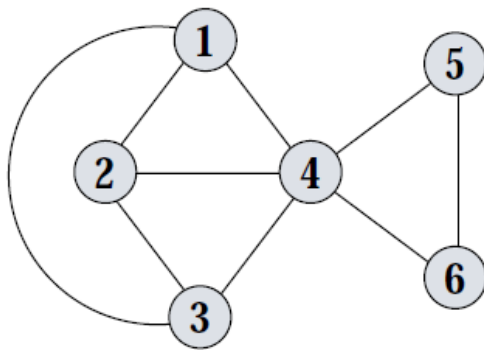
Für die Darstellung von Graphen $G = (V, E)$ gibt es zwei gängige Möglichkeiten:

- Bei lichten Graphen verwendet man gerne ***Adjazenzlisten***.
- Bei dichten Graphen verwendet man meist ***Adjazenzmatrizen***.

Graphen

Adjazenzlisten

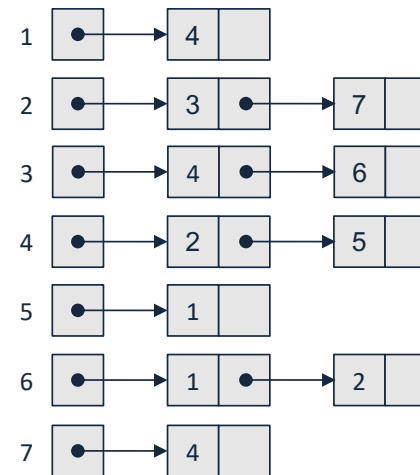
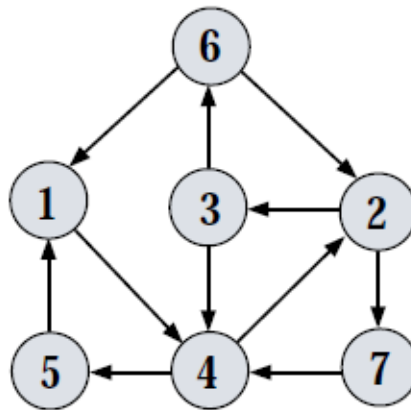
Die Adjazenzlisten-Darstellung eines **ungerichteten** Graphen $G = (V, E)$ besitzt für jeden Knoten $q \in V$ die Liste aller mit q verbundenen (adjazenten) Knoten.



Graphen

Adjazenzlisten

Die Adjazenzlisten-Darstellung eines **gerichteten** Graphen $G = (V, E)$ besitzt für jeden Knoten $q \in V$ die Liste aller direkten Nachfolger r von Knoten q , die also zu q adjazent sind.

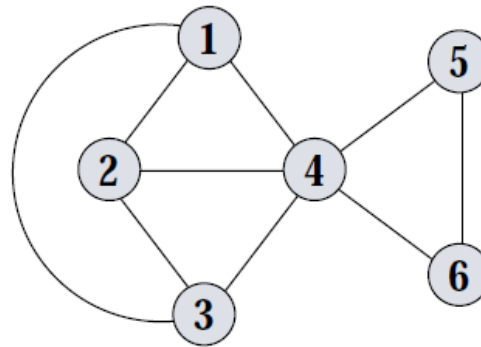


Graphen

Adjazenzmatrix

Die Adjazenzmatrix A eines Graphen $G = (V, E)$ ist eine $|V| \times |V|$ -Matrix mit den Elementen

$$a_{ij} = \begin{cases} 1 & \text{falls } j \text{ adjazent zu } i \\ 0 & \text{sonst} \end{cases}$$



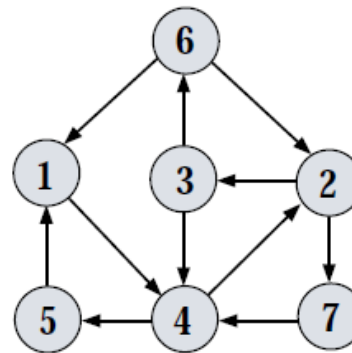
	1	2	3	4	5	6
1	0	1	1	1	0	0
2	1	0	1	1	0	0
3	1	1	0	1	0	0
4	1	1	1	0	1	1
5	0	0	0	1	0	1
6	0	0	0	1	1	0

Graphen

Adjazenzmatrix

Die Adjazenzmatrix A eines Graphen $G = (V, E)$ ist eine $|V| \times |V|$ -Matrix mit den Elementen

$$a_{ij} = \begin{cases} 1 & \text{falls } j \text{ adjazent zu } i \\ 0 & \text{sonst} \end{cases}$$



	1	2	3	4	5	6	7
1	0	0	0	1	0	0	0
2	0	0	1	0	0	0	1
3	0	0	0	1	0	1	0
4	0	1	0	0	1	0	0
5	1	0	0	0	0	0	0
6	1	1	0	0	0	0	0
7	0	0	0	1	0	0	0

Breitensuche

Breitensuche (breadth-first search - BFS)

Zielsetzung

Das Ziel der Breitensuche in einem Graphen ist es, ausgehend von einem Startknoten

- Alle **erreichbaren Knoten** zu identifizieren,
- für jeden erreichbaren Knoten die Distanz (Anzahl der beschrifteten Kanten) zum Start zu ermitteln (auch: **kürzester Pfad**) und
- diesen Pfad festzuhalten

Breitensuche (breadth-first search - BFS)

Vorgehensweise

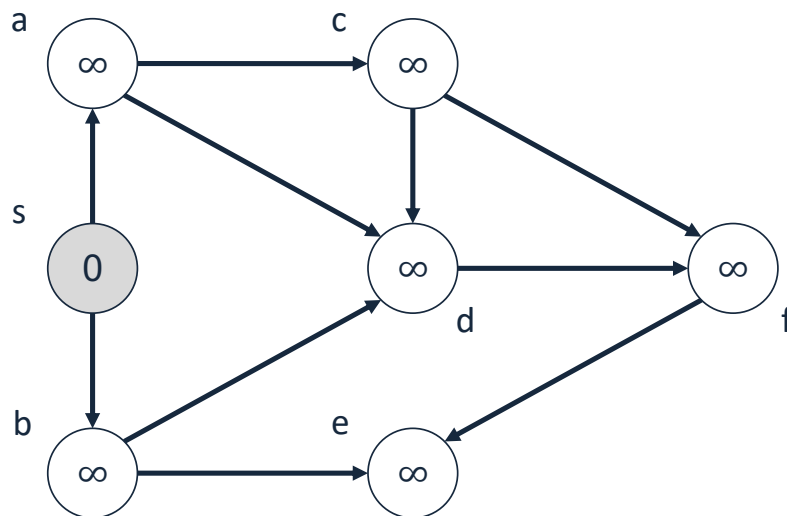
Die Knoten des Graphen werden in drei Kategorien eingeteilt:

- „Weiße“ Knoten sind noch nicht entdeckt
(am Anfang sind alle Knoten weiß)
- „Graue“ Knoten sind entdeckt, besitzen aber evtl. noch weiße Nachbarn
- „Schwarze“ Knoten sind vollständig bearbeitet und können nicht mehr zu unentdeckten Knoten führen

Lösungsidee: Die grauen Knoten werden in einer Queue verwaltet, die den Frontverlauf der Suche wiedergibt.

Breitensuche (breadth-first search - BFS)

Beispiel



Ausgangssituation:

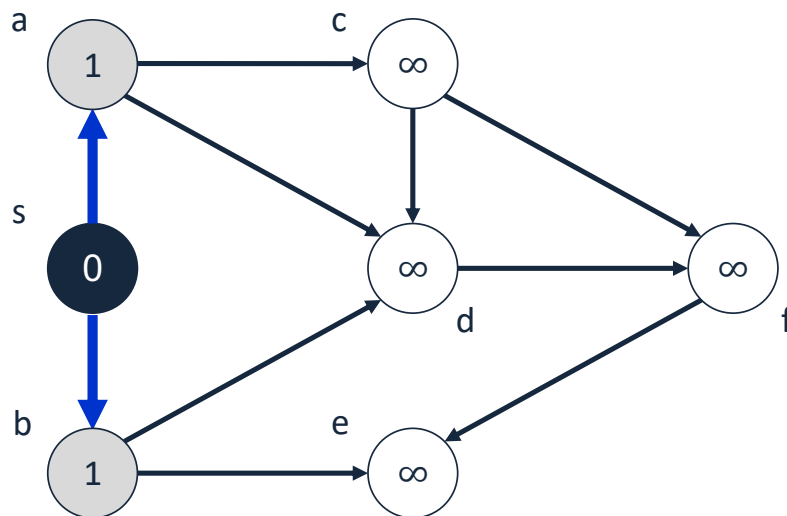
Nur der Startknoten ist grau, besitzt die Distanz 0 und steht in der Queue der zu bearbeitenden Knoten.

Alle anderen Knoten sind weiß und haben die Entfernung „unendlich“.

Queue: s

Breitensuche (breadth-first search - BFS)

Beispiel



Nächster Schritt:

Der vorderste Knoten der Queue wird entfernt und schwarz eingefärbt; alle davon erreichbaren Knoten grau eingefärbt und in die Queue eingefügt.

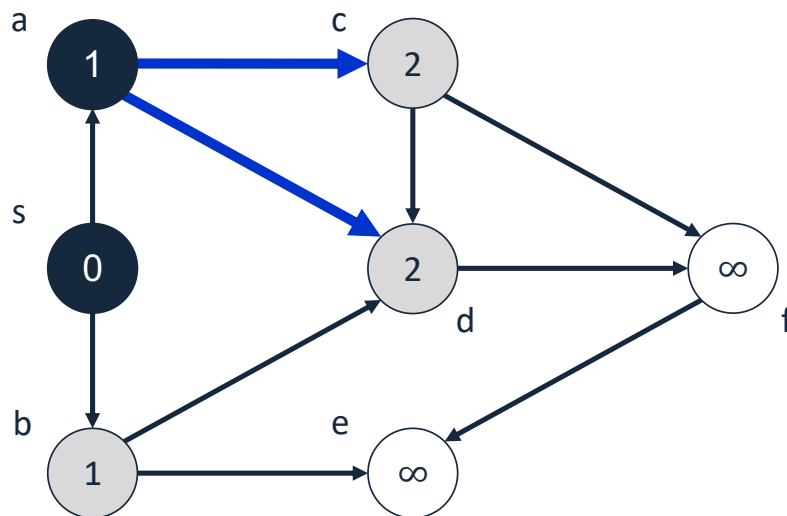
Außerdem wird für die neuen grauen Knoten die Distanz festgelegt.

Queue:

a	b
---	---

Breitensuche (breadth-first search - BFS)

Beispiel



Nächster Schritt:

Der vorderste Knoten der Queue wird entfernt und schwarz eingefärbt; alle davon erreichbaren Knoten grau eingefärbt und in die Queue eingefügt.

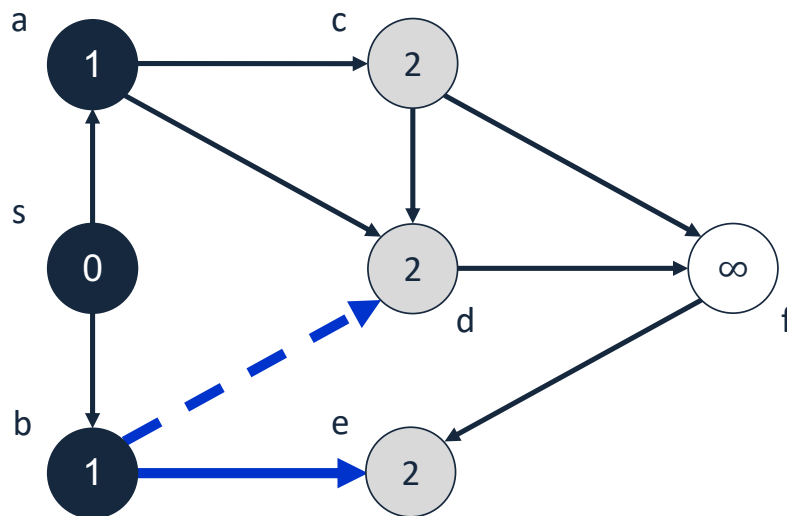
Außerdem wird für die neuen grauen Knoten die Distanz festgelegt.

Queue:

b	c	d
---	---	---

Breitensuche (breadth-first search - BFS)

Beispiel



Nächster Schritt:

Der vorderste Knoten der Queue wird entfernt und schwarz eingefärbt; alle davon erreichbaren Knoten grau eingefärbt und in die Queue eingefügt.

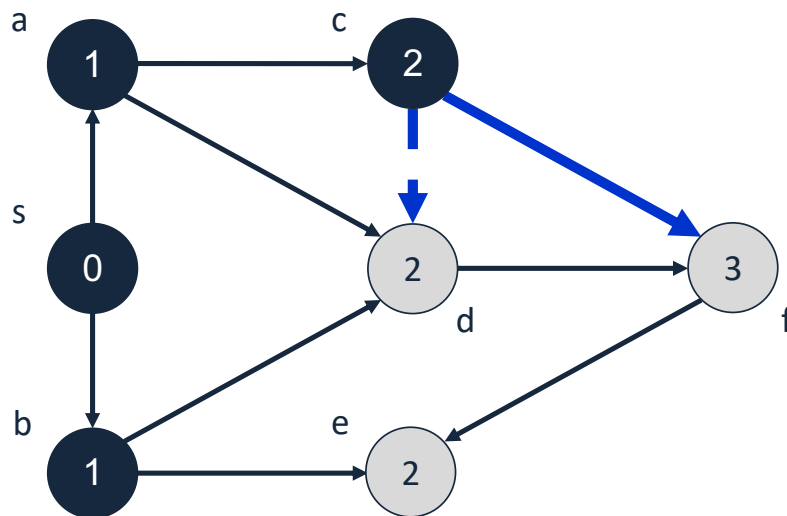
Außerdem wird für die neuen grauen Knoten die Distanz festgelegt.

Queue:

c	d	e
---	---	---

Breitensuche (breadth-first search - BFS)

Beispiel



Nächster Schritt:

Der vorderste Knoten der Queue wird entfernt und schwarz eingefärbt; alle davon erreichbaren Knoten grau eingefärbt und in die Queue eingefügt.

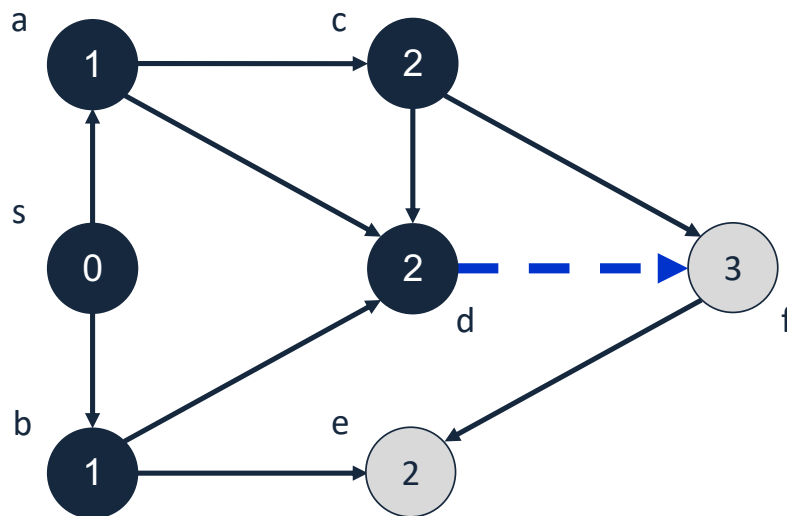
Außerdem wird für die neuen grauen Knoten die Distanz festgelegt.

Queue:

d	e	f
---	---	---

Breitensuche (breadth-first search - BFS)

Beispiel



Nächster Schritt:

Der vorderste Knoten der Queue wird entfernt und schwarz eingefärbt; alle davon erreichbaren Knoten grau eingefärbt und in die Queue eingefügt.

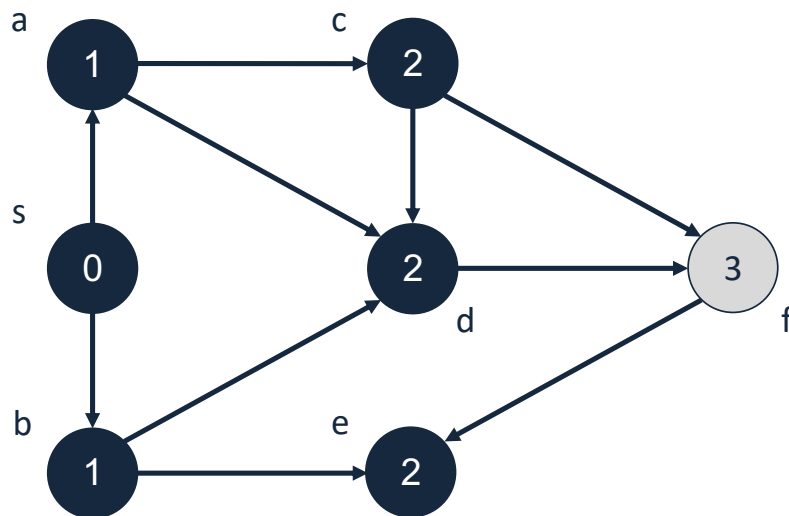
Außerdem wird für die neuen grauen Knoten die Distanz festgelegt.

Queue:

e	f
---	---

Breitensuche (breadth-first search - BFS)

Beispiel



Nächster Schritt:

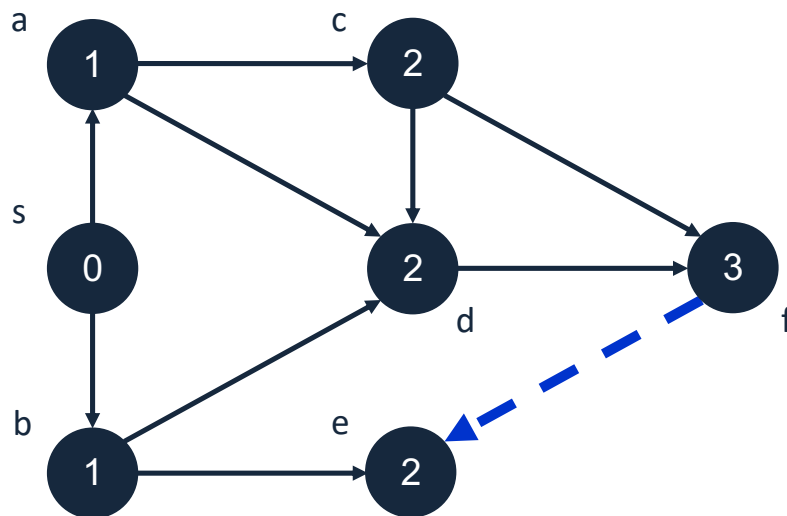
Der vorderste Knoten der Queue wird entfernt und schwarz eingefärbt; alle davon erreichbaren Knoten grau eingefärbt und in die Queue eingefügt.

Außerdem wird für die neuen grauen Knoten die Distanz festgelegt.

Queue: f

Breitensuche (breadth-first search - BFS)

Beispiel



Nächster Schritt:

Der vorderste Knoten der Queue wird entfernt und schwarz eingefärbt; alle davon erreichbaren Knoten grau eingefärbt und in die Queue eingefügt.

Außerdem wird für die neuen grauen Knoten die Distanz festgelegt.

Queue: leer

Breitensuche (breadth-first search - BFS)

Pseudocode

```

BFS( $V, E, s$ )
1: for each  $v \in V$  do
2:    $v.color \leftarrow \text{WHITE}$ 
3:    $v.dist \leftarrow \infty$ 
4:    $v.pred \leftarrow \text{NIL}$ 
5: end for
6:  $s.color \leftarrow \text{GRAY}$ 
7:  $s.dist \leftarrow 0$ 
8:  $queue \leftarrow \text{new Queue}$  ▷ FIFO Queue
9:  $queue.push(s)$ 
10: while  $queue$  not empty do
11:    $v \leftarrow queue.pop()$ 
12:   for each  $v \rightarrow u \in E$  do ▷ Alle von  $v$  ausgehenden Kanten
13:     if  $u.color = \text{WHITE}$  then
14:        $u.color \leftarrow \text{GRAY}$ 
15:        $u.dist \leftarrow v.dist + 1$ 
16:        $u.pred \leftarrow v$ 
17:        $queue.push(u)$ 
18:     end if
19:   end for
20:    $v.color \leftarrow \text{BLACK}$ 
21: end while

```

Breitensuche (breadth-first search - BFS)

Analyse

- Jeder Knoten wird genau einmal in die Queue aufgenommen und daraus wieder entfernt. Dieser Teil der Lösung besitzt die Zeitkomplexität $O(|V|)$.
- Jede Kante wird nur einmal betrachtet (nämlich beim Entfernen ausgehenden Knotens aus der Queue): $O(|E|)$
- In Summe ergibt sich für die Breitensuche eine lineare Zeitkomplexität von $O(|V| + |E|)$

Tiefensuche

Tiefensuche (depth-first search - DFS)

Vorgehen

- Wie bei der Breitensuche sollen alle erreichbaren Knoten ermittelt werden.
- Der nächste zu bearbeitenden Knoten wird allerdings nicht mit einer Queue festgelegt, sondern durch einen rekursiven Abstieg.
- Die Abbruchbedingung der Rekursion ist gegeben, wenn vom aktuellen Knoten kein weiterer unbesuchter Knoten erreicht werden kann.
- Die Färbungen „weiß“, „grau“ und „schwarz“ werden analog zur Breitensuche verwendet.
- Anders als bei der Breitensuche wird nicht die Distanz zum Startknoten identifiziert. Dagegen erhält jeder Knoten zwei Zeitmarken:
 - Zeitpunkt, zu dem dieser Knoten „grau“ wurde.
 - Zeitpunkt, zu dem dieser Knoten „schwarz“ wurde

Tiefensuche (depth-first search - DFS)

Pseudocode

DFS(V, E)

```

1: for each  $v \in V$  do
2:    $v.color \leftarrow \text{WHITE}$ 
3:    $v.pred \leftarrow \text{NIL}$ 
4: end for
5:  $time \leftarrow 0$  ▷ Globale Zeit auf 0
6: for each  $v \in V$  do
7:   if  $v.color = \text{WHITE}$  then
8:     DFS-SUBTREE( $V, E, v$ )
9:   end if
10: end for

```

DFS-SUBTREE(V, E, v)

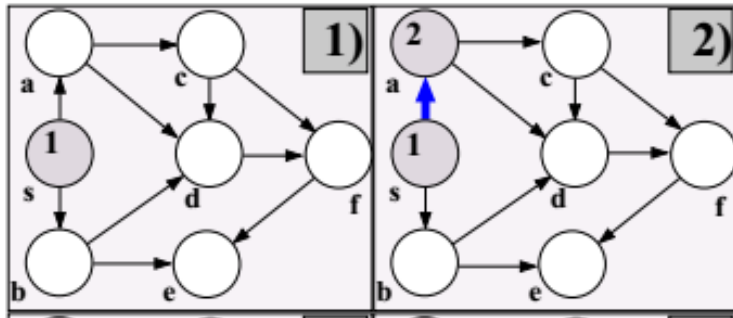
```

1:  $v.color \leftarrow \text{GRAY}$ 
2:  $time \leftarrow time + 1$  ▷ Globale Zeit erhöhen
3:  $v.begin \leftarrow time$ 
4: for each  $v \rightarrow u \in E$  do ▷ Alle von  $v$  ausgehenden Kanten
5:   if  $u.color = \text{WHITE}$  then
6:      $u.pred \leftarrow v$ 
7:     DFS-SUBTREE( $V, E, u$ )
8:   end if
9: end for
10:  $v.color \leftarrow \text{BLACK}$ 
11:  $time \leftarrow time + 1$  ▷ Globale Zeit erhöhen
12:  $v.end \leftarrow time$ 

```

Tiefensuche (depth-first search - DFS)

Beispiel



DFS-SUBTREE(V, E, v)

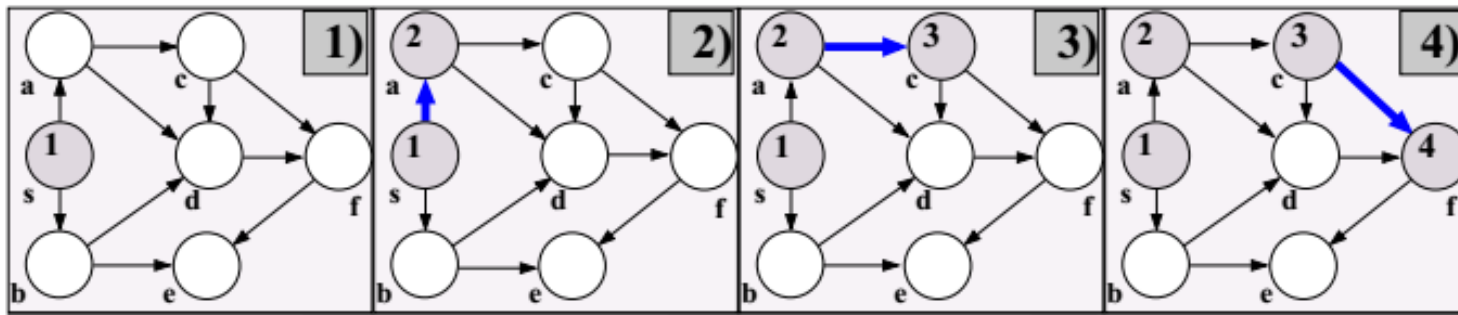
```

1:  $v.color \leftarrow \text{GRAY}$ 
2:  $time \leftarrow time + 1$  ▷ Globale Zeit erhöhen
3:  $v.begin \leftarrow time$ 
4: for each  $v \rightarrow u \in E$  do ▷ Alle von  $v$  ausgehenden Kanten
5:   if  $u.color = \text{WHITE}$  then
6:      $u.pred \leftarrow v$ 
7:     DFS-SUBTREE( $V, E, u$ )
8:   end if
9: end for
10:  $v.color \leftarrow \text{BLACK}$ 
11:  $time \leftarrow time + 1$  ▷ Globale Zeit erhöhen
12:  $v.end \leftarrow time$ 

```

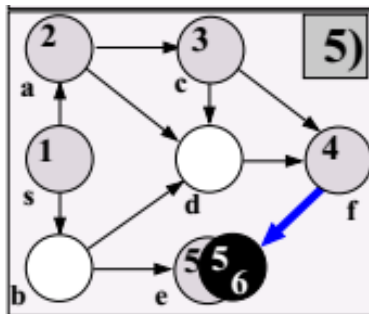
Tiefensuche (depth-first search - DFS)

Beispiel



Tiefensuche (depth-first search - DFS)

Beispiel



DFS-SUBTREE(V, E, v)

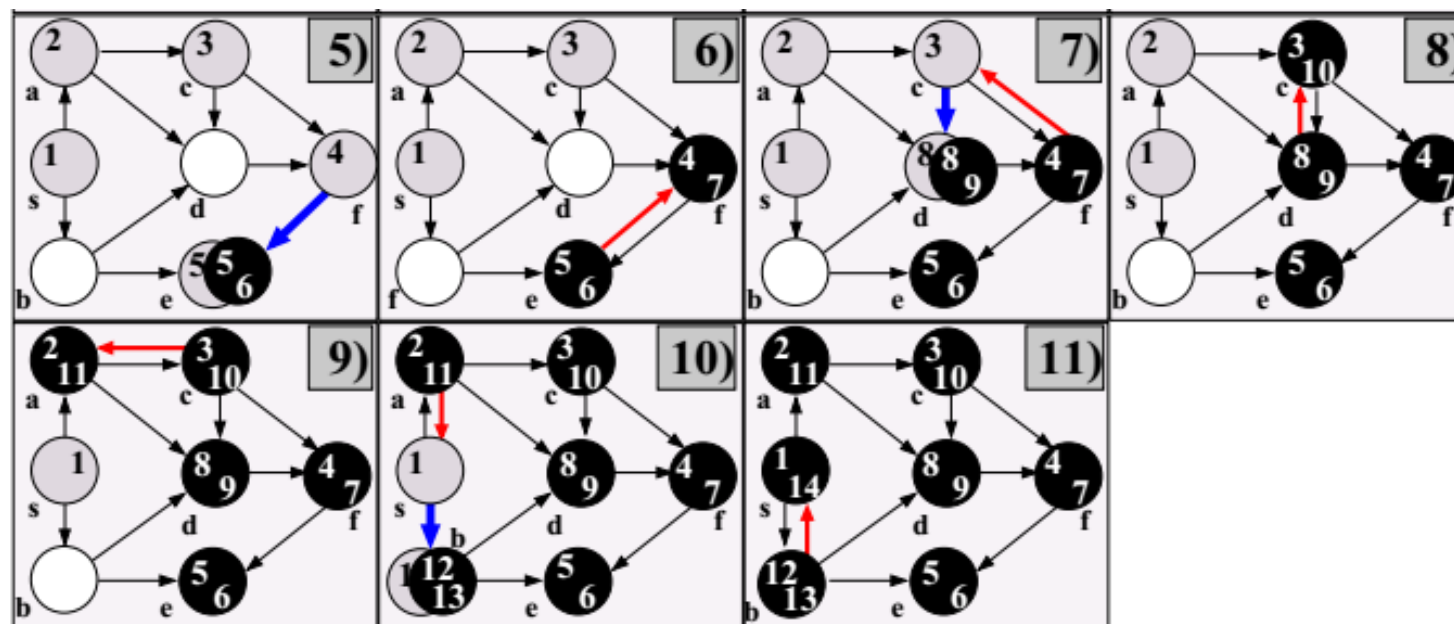
```

1:  $v.color \leftarrow \text{GRAY}$ 
2:  $time \leftarrow time + 1$                                 ▷ Globale Zeit erhöhen
3:  $v.begin \leftarrow time$ 
4: for each  $v \rightarrow u \in E$  do                                ▷ Alle von  $v$  ausgehenden Kanten
5:   if  $u.color = \text{WHITE}$  then
6:      $u.pred \leftarrow v$ 
7:     DFS-SUBTREE( $V, E, u$ )
8:   end if
9: end for
10:  $v.color \leftarrow \text{BLACK}$ 
11:  $time \leftarrow time + 1$                                 ▷ Globale Zeit erhöhen
12:  $v.end \leftarrow time$ 

```

Tiefensuche (depth-first search - DFS)

Beispiel



Tiefensuche (depth-first search - DFS)

Analyse

- Jeder Knoten wird im Rahmen der Rekursion genau einmal besucht. Dieser Teil der Lösung besitzt die Zeitkomplexität $O(|V|)$.
- Jede Kante wird nur einmal betrachtet (nämlich beim ersten Besuch des ausgehenden Knoten): $O(|E|)$
- In Summe ergibt sich auch für die Tiefensuche eine lineare Zeitkomplexität von $O(|V| + |E|)$

Tiefensuche (depth-first search - DFS)

Topologisches Sortieren

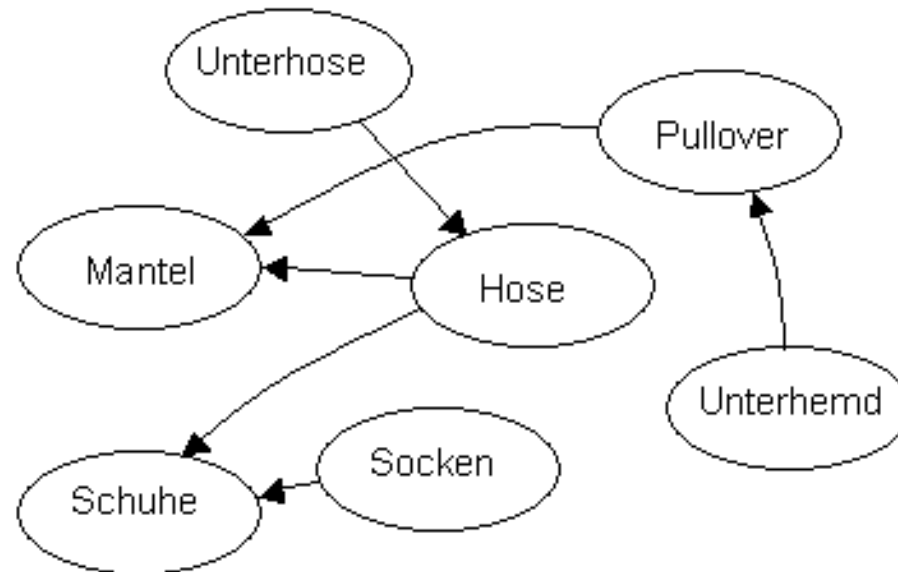
- Die Tiefensuche liefert mit ihren Zeitmarken Strukturinformationen über den Graphen, die für weitergehende Algorithmen genutzt werden können.
- Ein Anwendungsfall ist das Topologische Sortieren, bei dem alle Knoten des Graphen in eine Reihenfolge gebracht werden.
- Bei dieser Reihenfolge ist sichergestellt, dass

$$(u, v) \in E \rightarrow u < v$$

Tiefensuche (depth-first search - DFS)

Topologisches Sortieren - Beispiel

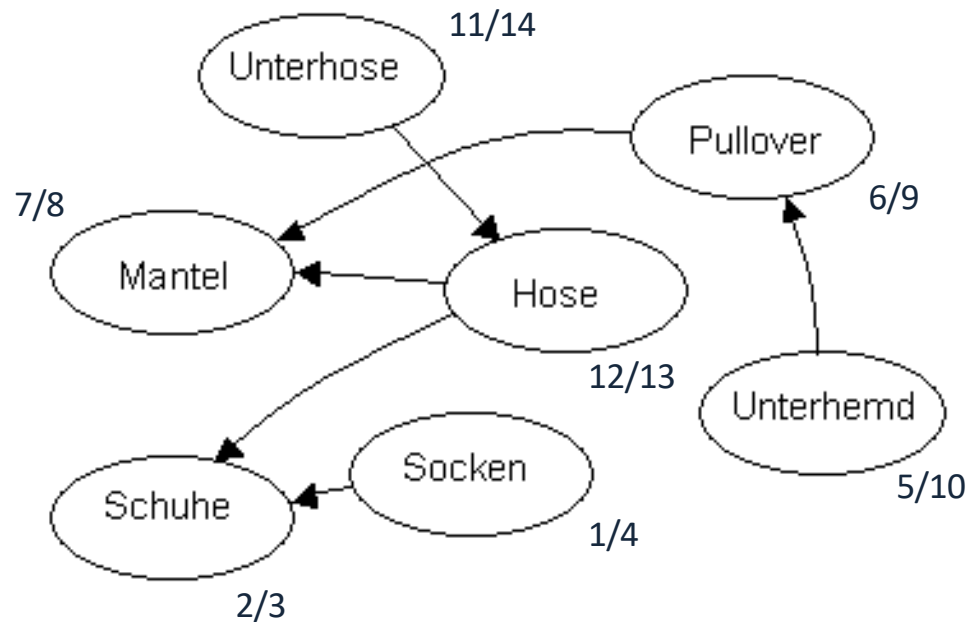
Gegeben ist ein azyklischer gerichteter Graph, der die Abhängigkeiten beim Ankleiden wiedergibt.



Tiefensuche (depth-first search - DFS)

Topologisches Sortieren - Beispiel

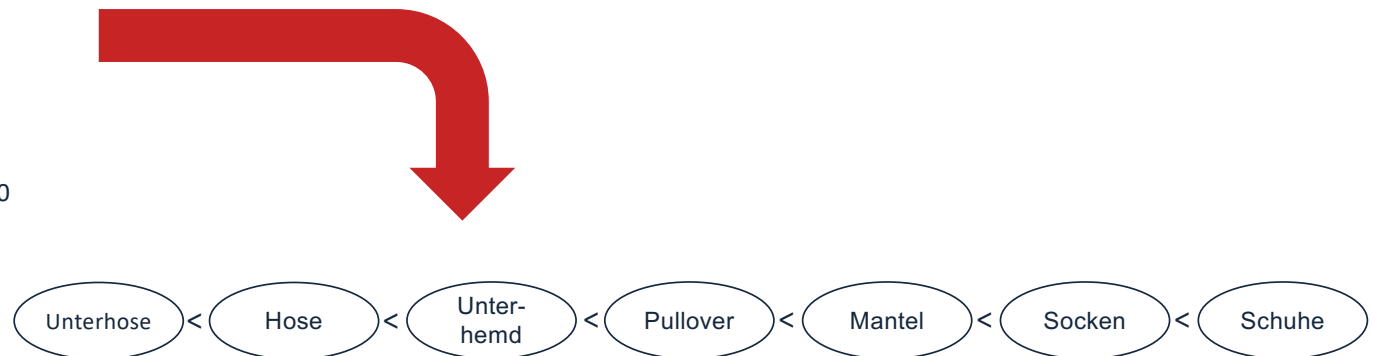
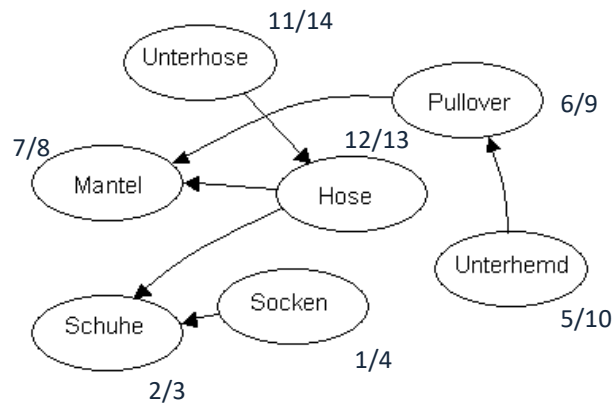
Nach einer Tiefensuche besitzt jeder Knoten zwei Zeitmarken (Zeitpunkt „grau“ und Zeitpunkt „schwarz“)



Tiefensuche (depth-first search - DFS)

Topologisches Sortieren - Beispiel

Eine mögliche topologische Sortierung ist nun die Anordnung nach den „schwarz“-Zeitmarken in absteigender Reihenfolge



Vielen Dank für die Aufmerksamkeit!